

Object-Oriented Programming (OOP) Tasks

Objective

The objective of this task is to build a strong foundation in Object-Oriented Programming (OOP) by applying its core concepts in a practical project, implemented using clean code and published on GitHub.

The project can be implemented either as a Jupyter Notebook or as multiple Python files, allowing flexibility based on the learner's current level.

Tasks

Task 1: What is Object-Oriented Programming (OOP)?

Object-Oriented Programming (OOP) is a programming paradigm that organizes software around objects rather than functions and logic.

Task 2: Difference between Class and Object

a class is a conceptual blueprint or template, while an object is a tangible instance created from that blueprint that exists in a computer's memory

Task 3: Difference between Encapsulation and Abstraction

abstraction focuses on what an object does, while encapsulation focuses on safeguarding how it stores data

Task 4: Difference between Inheritance and Composition

Inheritance models an "is-a" relationship (like a Dog is-an Animal), where a child class extends a parent class. Composition models a "has-a" relationship (like a Car has-an Engine), where a class contains objects of other classes as fields or properties

Task 5: Difference between Method Overloading and Method

Overriding

Method overloading happens when a class has multiple methods with the same name but different parameters. Method overriding happens when a child class writes a new version of a method that already exists in its parent class using the exact same name and parameters

Task 6: Difference between Compile-Time and Run-Time Polymorphism

Compile-time is faster, while runtime offers more flexibility

Task 7: Difference between Abstract Class and Interface

an abstract class defines a baseline identity and shared behavior (what an object is), while an interface defines a contract for behavior (what an object can do)

Task 8: Difference between Association, Aggregation, and Composition (*Conceptual only*)

Association is a general link where two objects use each other but

live separately. Aggregation is a "has-a" link with weak ownership. Composition is a strict "part-of" link where parts cannot live without the whole

Task 9: Difference between Static Members and Instance Members

static members belong to the class itself, while instance members belong to individual objects (instances) created from that class

Task 10: Difference between Constructor and Destructor

A constructor is a special function that runs automatically when an object is created to set initial values. A destructor is a special function that runs automatically when an object is destroyed to clean up resources

Task 11: Difference between Access Modifiers

- **Public modifier:** Accessible from anywhere in the program
- **Protected modifier:** Accessible within the same package and by subclasses
- **Private modifier:** Accessible only within the same class
- **Default modifier:** Accessible only within the same package

Task 12: Difference between Mutable and Immutable Objects

Mutable class objects are those whose state can be modified after initialization

Task 13: Difference between Shallow Copy and Deep Copy

Immutable class objects are those whose state cannot be modified after initialization.

Task 14: Difference between Tight Coupling and Loose Coupling

Tight coupling means software components are heavily dependent on each other and

share deep internal knowledge. Loose coupling means components are independent and interact through abstract interfaces, allowing you to change or replace one part without breaking the rest.

Task 15: Difference between Dependency Injection and Direct Dependency (*Basic concept only*)

Difference between Dependency Injection and Direct Dependency

Task 16: Difference between OOP and Procedural Programming

while OOP binds data and behaviors together into independent units called objects

Task 17: Difference between OOP and Functional Programming

Object-Oriented Programming (OOP) binds data and its related behavior together into objects, while Functional Programming (FP) treats data and behavior as completely separate entities

Task 18: Difference between Runtime Binding and Compile-Time Binding

Compile-time is faster, while runtime is more flexible

Task 19: Difference between Multiple Inheritance and Multilevel Inheritance

involves a single child class inheriting directly from **two or more distinct parent**

classes simultaneously, while **Multilevel Inheritance** involves a **linear chain of**

classes where a child class inherits from a parent class, which in turn inherits from another grandparent class

Task 20: Difference between Garbage Collection and Manual Memory Management

Garbage collection is an automated system where the runtime environment tracks and cleans up unused memory. In contrast, manual memory management requires the programmer to explicitly allocate and free memory using specific commands, offering total control at the risk of critical safety errors

Task 21: OOP Project (Mandatory – GitHub Based)

Project Objective

Develop a **basic-to-intermediate OOP project** using core OOP concepts only and upload it to **GitHub** with proper documentation.

Project Implementation Options

The project can be implemented in **one of the following ways**:

Option 1: Jupyter Notebook (Allowed)

- Single .ipynb file
- Well-organized sections
- Clear markdown explanations

Option 2: Multiple Python Files (Bonus)

- Project divided into logical modules
- Better code organization
- Closer to real-world development practices

Suggested Project Ideas

- Student Management System
- Library Management System
- Banking System
- Hospital Management System
- Inventory Management System

Project Requirements (Mandatory)

Regardless of the chosen option, the project must include:

- Classes & Objects
- Encapsulation
- Abstraction
- Inheritance
- Polymorphism
- Constructors
- Exception handling
- Clean and readable code
- GitHub repository

Task 22: GitHub Repository

Requirements Repository Must

Include:

- Project source code (.ipynb or .py files)
- README.md
- .gitignore (optional but recommended)
- Clear commit history

README.md Requirements

The README.md file must contain:

- Project title
- Project description
- OOP concepts used
- Project features
- How to run the project
- Implementation type:
 - Notebook OR
 - Multi-file project (Bonus)
- Author name

README.md Example (Snippet)

Implementation

This project is implemented as a Jupyter Notebook.
(The multi-file implementation is considered a bonus.)